Marketplace API Documentation

This project is a Flask-based RESTful API that simulates a simple online Marketplace system. It allows interaction with two main resources: Users (login and user listing) and Items (retrieving and adding items). The main entry point (`main.py`) initializes the Flask app, enables CORS, integrates Swagger for API documentation, and registers the blueprints.

1. main.py — Application Entry Point

The main script initializes the Flask web application, configures global settings, registers blueprints, and defines the home route. It uses Flask, Flask-CORS for cross-origin requests, and Flasgger for Swagger UI documentation.

- Initializes Flask app and enables CORS & Swagger.
- Registers blueprints for users and items.
- Defines '/' route to confirm server status.
- Runs app in debug mode for development.

2. user.py — User Management Module

Handles user-related operations like listing users and logging in. Blueprints modularize user endpoints, while jsonify and request manage JSON communication.

- GET /users — returns mock list of users.
- POST /login — checks username/password and returns mock JWT token if valid.

3. items.py — Item Management Module

Manages item-related functionalities: listing items, fetching individual items, and adding new items. Each endpoint simulates database-like interactions using mock data.

- GET /items — returns a list of items.
- GET /items/ — returns details of a single item.
- POST /items — adds a new item to the marketplace.

System Architecture Summary

The project follows a modular architecture with separate blueprints for each domain. `main.py` integrates them and runs the Flask server, enabling a scalable, organized API structure.

- main.py — API setup and route registration.
- user.py — Handles authentication and user listing.
- items.py — Handles product-related endpoints.

Future Improvements

To make this project production-ready, consider implementing the following:

- Integrate a real database (MySQL, SQLite, MongoDB).
- Add JWT-based authentication with token expiration.
- Implement input validation and error handling.

- Enhance Swagger documentation for all endpoints.